

Rapport de TP 1 - Analyse statique (SAST) et sécurité des dépendances (SCA)

Université : INSAT

Classe : GL

Année universitaire : 2026/2027

Équipe :

- Ahmed Jammoussi
- Mohamed Taher Ben Hassine
- Yassine Sallemi
- Youssef Fadhloun

1. Comprendre le « shift-left »

En sécurité classique, on teste souvent **après** le développement (voire en production). Le *shift-left* consiste à déplacer les contrôles **vers la gauche** du cycle de vie : dès l'écriture du code et de la déclaration des dépendances.

Dans ce TP, cela signifie :

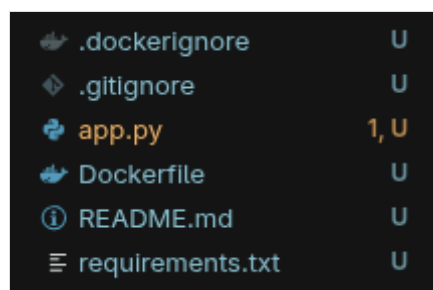
1. écrire / lancer l'application localement (Docker) ;
2. scanner le **code source** (SAST) ;
3. scanner les **dépendances** (SCA) ;
4. corriger ;
5. **re-scanner** pour prouver que les alertes ont disparu.

Flux : Code vulnérable → Bandit (SAST) / Dependency-Check (SCA) → Correctifs → Re-scan avant/après → Démonstration.

2. Mise en place de l'environnement

2.1 Structure du dépôt

Application Python Flask minimale, un `Dockerfile`, un `requirements.txt`, puis les dossiers `reports/` et `screenshots/`.



 <code>.dockerignore</code>	U
 <code>.gitignore</code>	U
 <code>app.py</code>	1, U
 <code>Dockerfile</code>	U
 <code>README.md</code>	U
 <code>requirements.txt</code>	U

```

"""Vulnerable Notes API – intentionally insecure.
"""

import sqlite3

from flask import Flask, request
from markupsafe import Markup

app = Flask(__name__)

DB = "users.db"

def init_db():
    conn = sqlite3.connect(DB)
    conn.execute(
        "CREATE TABLE IF NOT EXISTS users "
        "(id INTEGER PRIMARY KEY, name TEXT, password TEXT)"
    )
    conn.execute("DELETE FROM users")
    conn.execute(
        "INSERT INTO users (name, password) VALUES (?, ?)",
        ("admin", "admin123"),
    )
    conn.execute(
        "INSERT INTO users (name, password) VALUES (?, ?)",
        ("alice", "alice123"),
    )
    conn.commit()
    conn.close()

@app.route("/")
def index():
    return (
        "Vulnerable Notes API – endpoints: "
        "/user?name=... , /hello?name=..."
    )

# VULN 1 – SQL injection (Bandit B608): user input is concatenated into SQL.
@app.route("/user")
def get_user():
    name = request.args.get("name", "")
    conn = sqlite3.connect(DB)
    query = "SELECT id, name FROM users WHERE name = '" + name + "'"
    rows = conn.execute(query).fetchall()
    conn.close()
    return {"users": rows}

# VULN 2 – reflected XSS (Bandit B704): Markup marks untrusted input as safe HTML.
@app.route("/hello")
def hello():
    name = request.args.get("name", "inconnu")
    return Markup(f"<h1>Bonjour {name}</h1>")

if __name__ == "__main__":
    init_db()

```

2.2 Image Docker

Commandes utilisées :

```
docker build -t vulnerable-notes .
docker run -d --rm -p 5000:5000 --name notes vulnerable-notes
```

```
• (base) mtbh@fedora:~/work/fac/gl5s1/DevSecOps/TPs/DevSecOps-Tp1$ docker build -t vulnerable-notes .
[+] Building 0.3s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 263B                             0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 0.0s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 219B                                  0.0s
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:0dd364ba7e10242f07755449e3a3d0e35f9efd987952737b90def6 0.0s
=> => resolve docker.io/library/python:3.11-slim@sha256:0dd364ba7e10242f07755449e3a3d0e35f9efd987952737b90def6 0.0s
=> [internal] load build context                                0.0s
=> => transferring context: 181B                                  0.0s
=> CACHED [2/5] WORKDIR /app                                    0.0s
=> CACHED [3/5] COPY requirements.txt .                          0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY app.py .                                   0.0s
=> exporting to image                                           0.1s
=> => exporting layers                                           0.0s
=> => exporting manifest sha256:11ec6b6e4cc045364b89449b7ac6d18f695fad33410df0b3a065d248bd796f4f 0.0s
=> => exporting config sha256:fe0cc5b84d2cf828d4d06b0652129f845c1c52850fd6c18c57df2212d28d7682 0.0s
=> => exporting attestation manifest sha256:3957ce073a5f6838fd4c969878aca76989061cc21c2be388622200a90ade8e00 0.0s
=> => exporting manifest list sha256:cca64ab37735ff3ab54f3c77af3ce0f9aab5bfc71457c9f8d25cb3277899b6c1 0.0s
=> => naming to docker.io/library/vulnerable-notes:latest      0.0s
=> => unpacking to docker.io/library/vulnerable-notes:latest   0.0s
```

```
• (base) mtbh@fedora:~/work/fac/gl5s1/DevSecOps/TPs/DevSecOps-Tp1$ docker run -d --rm -p 5000:5000 --name notes vulnerable-notes
ddb210ebc6a1889aad841c057c3e268585b0738533b07858fd797e70ac5172de
```

2.3 Reproduction des failles (avant correctifs)

Lookup normal - seul admin est renvoyé :

```
curl -s --get "http://localhost:5000/user" --data-urlencode "name=admin"
```

```
• DevSecOps-Tp1 $ curl -s --get "http://localhost:5000/user" --data-urlencode "name=admin"
{"users":[[1,"admin"]]}
```

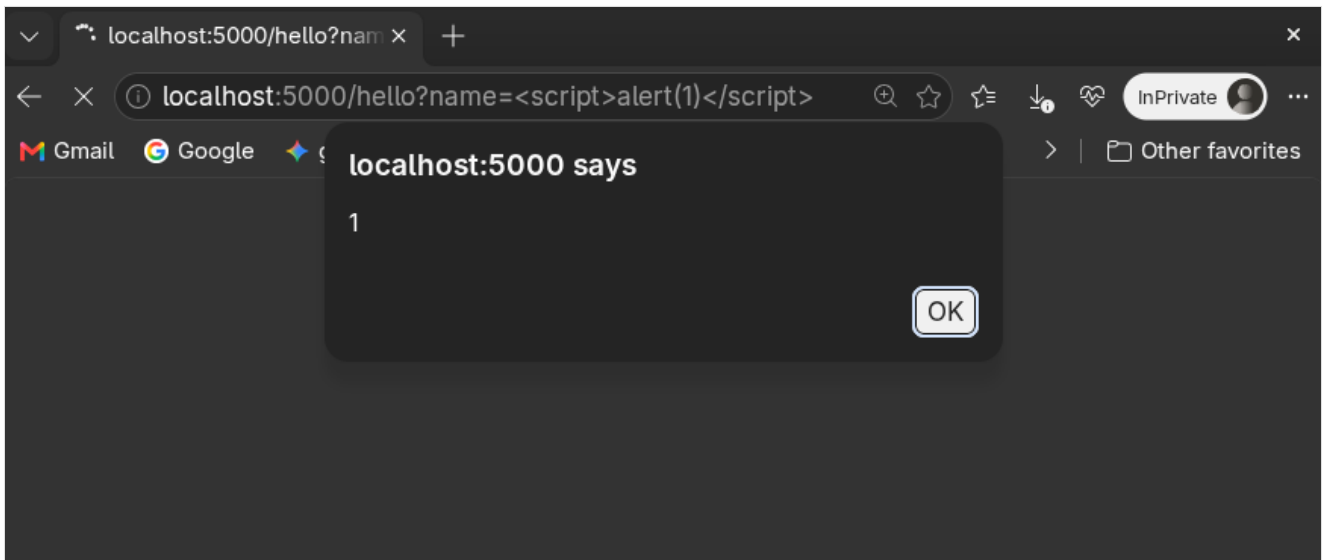
Injection SQL - la charge ' OR '1'='1 renvoie **tous** les utilisateurs (admin et alice) :

```
curl -s --get "http://localhost:5000/user" --data-urlencode "name=' OR '1'='1"
```

```
• DevSecOps-Tp1 $ curl -s --get "http://localhost:5000/user" --data-urlencode "name=' OR '1'='1"
{"users":[[1,"admin"],[2,"alice"]]}
```

XSS réfléchi - le navigateur / curl reçoit le <script> **non échappé** :

```
curl -s --get "http://localhost:5000/hello" \
--data-urlencode "name=<script>alert(1)</script>"
```



```
DevSecOps-Tp1 $ curl -s --get "http://localhost:5000/hello" --data-urlencode "name=<script>alert(1)</script>"
<h1>Bonjour <script>alert(1)</script></h1>DevSecOps-Tp1 $
```

Pourquoi c'est grave :

Faible	Cause dans le code	Impact
Injection SQL	Concaténation de name dans la requête SQL	Lecture (voire modification) de données non autorisées
XSS	markupsafe.Markup sur une entrée utilisateur	Exécution de JavaScript dans le navigateur de la victime

3. Analyse statique (SAST) avec Bandit

3.1 Lancement du scan

```
mkdir -p reports

docker run --rm -v "$PWD":/src python:3.11-slim sh -c \
  "pip install -q bandit && bandit -r /src/app.py -f txt" \
  | tee reports/bandit-avant.txt
```

```
DevSecOps-Tp1 $ docker run --rm -v "$PWD":/src python:3.11-slim sh -c \
  "pip install -q bandit && bandit -r /src/app.py -f txt" \
  | tee reports/bandit-avant.txt
```

3.2 Rapport Bandit - AVANT correctifs

Fichier : reports/bandit-avant.txt

```
Run started:2026-10-07 22:40:59.198539+00:00

Test results:
```

```

>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based
query construction.
Severity: Medium    Confidence: Low
CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/
b608_hardcoded_sql_expressions.html
Location: /src/app.py:48:12
47     conn = sqlite3.connect(DB)
48     query = "SELECT id, name FROM users WHERE name = '" + name + "'"
49     rows = conn.execute(query).fetchall()

-----

>> Issue: [B704:markupsafe_markup_xss] Potential XSS with ``markupsafe.Markup`` detected. Do
not use ``Markup`` on untrusted data.
Severity: Medium    Confidence: High
CWE: CWE-79 (https://cwe.mitre.org/data/definitions/79.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/
b704_markupsafe_markup_xss.html
Location: /src/app.py:58:11
57     name = request.args.get("name", "inconnu")
58     return Markup(f"<h1>Bonjour {name}</h1>")
59

-----

>> Issue: [B104:hardcoded_bind_all_interfaces] Possible binding to all interfaces.
Severity: Medium    Confidence: Medium
CWE: CWE-605 (https://cwe.mitre.org/data/definitions/605.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/
b104_hardcoded_bind_all_interfaces.html
Location: /src/app.py:63:17
62     init_db()
63     app.run(host="0.0.0.0", port=5000, debug=False)

-----

Code scanned:
  Total lines of code: 46
  Total lines skipped (#nosec): 0
  Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
  Total issues (by severity):
    Undefined: 0
    Low: 0
    Medium: 3
    High: 0
  Total issues (by confidence):
    Undefined: 0
    Low: 1
    Medium: 1
    High: 1
Files skipped (0):

```

3.3 Interprétation des alertes

ID Bandit	CWE	Signification	Mission TP
B608	CWE-89	Injection SQL probable	Oui corrigée

ID Bandit	CWE	Signification	Mission TP
B704	CWE-79	XSS via Markup	Oui corrigée
B104	CWE-605	Écoute sur toutes les interfaces (0.0.0.0)	Traité (hôte via variable d'environnement)

Limite de Bandit : c'est un analyseur de motifs / règles, pas un moteur de flux de données complet. Il signale des *patterns* dangereux ; la preuve d'exploitabilité vient aussi des tests curl ci-dessus.

3.4 Correctifs appliqués

Diff des changements dans app.py :

```

42- # VULN 1 - SQL injection (Bandit B608): user input is concat
43- @app.route("/user")
44- def get_user():
45-     name = request.args.get("name", "")
46-     conn = sqlite3.connect(DB)
47-     query = "SELECT id, name FROM users WHERE name = '" + name + "'"
48-
49-     rows = conn.execute(query).fetchall()
50-     conn.close()
51-     return {"users": rows}
52-
53- # VULN 2 - reflected XSS (Bandit B704): Markup marks untrusted
54- @app.route("/hello")
55- def hello():
56-     name = request.args.get("name", "inconnu")
57-     return Markup(f"<h1>Bonjour {name}</h1>")
58-
59-
60-
61- if __name__ == "__main__":
62-     init_db()
63-
64-     app.run(host="0.0.0.0", port=5000, debug=False)
65-
43+ # FIX 1 - parameterized query (no string concatenation)
44+ @app.route("/user")
45+ def get_user():
46+     name = request.args.get("name", "")
47+     conn = sqlite3.connect(DB)
48+     rows = conn.execute(
49+         "SELECT id, name FROM users WHERE name = ?",
50+         (name,),
51+     ).fetchall()
52+     conn.close()
53+     return {"users": rows}
54-
55- # FIX 2 - escape untrusted input before returning HTML
56- @app.route("/hello")
57- def hello():
58+     name = request.args.get("name", "inconnu")
59+     return f"<h1>Bonjour {escape(name)}</h1>"
60-
61-
62-
63-
64- if __name__ == "__main__":
65-     init_db()
66+     # Bind address from env so Docker can set 0.0.0.0 without /
67+     # (avoids Bandit B104 in application code).
68+     host = os.environ.get("APP_HOST", "127.0.0.1")
69+     app.run(host=host, port=5000, debug=False)
70-

```

Injection SQL - avant :

```

query = "SELECT id, name FROM users WHERE name = '" + name + "'"
rows = conn.execute(query).fetchall()

```

Injection SQL - après (requête paramétrée) :

```

rows = conn.execute(
    "SELECT id, name FROM users WHERE name = ?",
    (name,),
).fetchall()

```

XSS - avant :

```

return Markup(f"<h1>Bonjour {name}</h1>")

```

XSS - après (échappement) :

```
from markupsafe import escape
return f"<h1>Bonjour {escape(name)}</h1>"
```

Bind 0.0.0.0 : le code lit APP_HOST (défaut 127.0.0.1). Dans le Dockerfile, ENV APP_HOST=0.0.0.0 est nécessaire pour que le port publié par Docker (-p 5000:5000) fonctionne. On évite ainsi un bind-all **codé en dur** dans le source, tout en gardant un comportement adapté au conteneur.

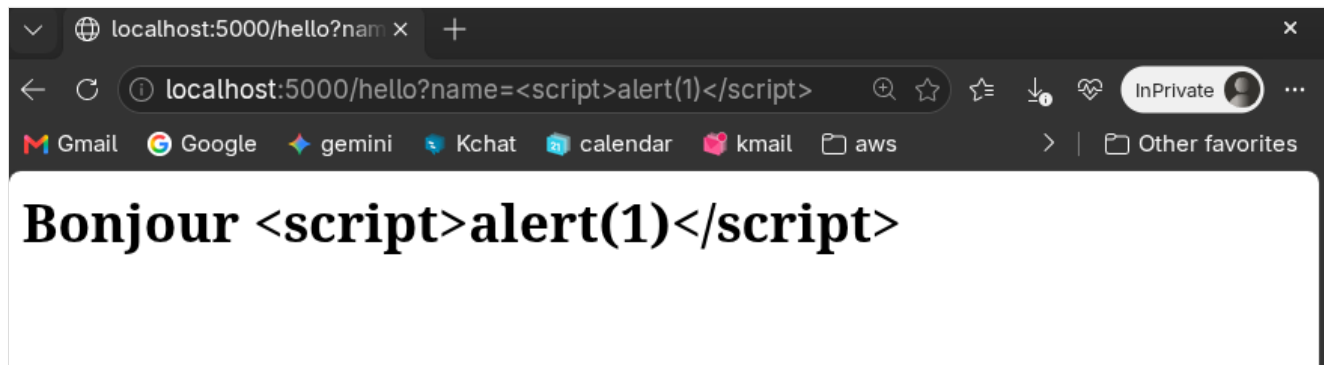
3.5 Vérification fonctionnelle après correctifs

```
docker build -t vulnerable-notes .
docker run -d --rm -p 5000:5000 --name notes vulnerable-notes
```

- La charge SQL ne renvoie plus tous les utilisateurs.
- Le XSS apparaît échappé (<script>...).

```
DevSecOps-Tp1 $ curl -s --get "http://localhost:5000/user" --data-urlencode "name=' OR '1'='1'" {"users": []}
```

```
DevSecOps-Tp1 $ curl -s --get "http://localhost:5000/hello" \
--data-urlencode "name=<script>alert(1)</script>"
<h1>Bonjour &lt;script&gt;alert(1)&lt;/script&gt;</h1>DevSecOps-Tp1 $
```



3.6 Rapport Bandit - APRÈS correctifs

Commande :

```
docker run --rm -v "$PWD":/src python:3.11-slim sh -c \
"pip install -q bandit && bandit -r /src/app.py -f txt" \
| tee reports/bandit-apres.txt
```

Fichier : reports/bandit-apres.txt

Run started:2026-10-07 23:31:07.846697+00:00

Test results:

No issues identified.

Code scanned:

Total lines of code: 50

Total lines skipped (#nosec): 0

```
Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0
```

Run metrics:

Total issues (by severity):

Undefined: 0

Low: 0

Medium: 0

High: 0

Total issues (by confidence):

Undefined: 0

Low: 0

Medium: 0

High: 0

Files skipped (0):

Résultat : les alertes B608, B704 et B104 ne sont plus signalées.

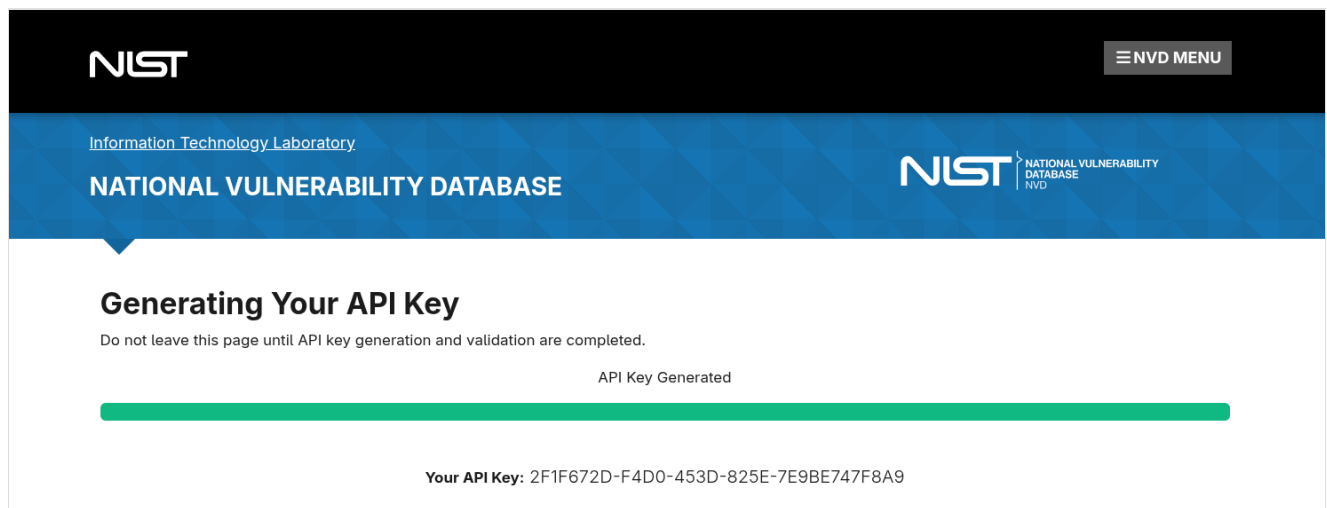
4. Analyse des dépendances (SCA) avec OWASP Dependency-Check

4.1 Clé API NVD

Sans clé, le premier téléchargement de la base NVD est très long.

Création d'une clé sur le site NVD, puis export local (jamais committée dans Git) :

```
export NVD_API_KEY="votre-clé"
```



The screenshot shows the NIST National Vulnerability Database (NVD) website. The header includes the NIST logo and a menu button. The main content area is titled "Generating Your API Key" and includes a progress bar and the generated API key: 2F1F672D-F4D0-453D-825E-7E9BE747F8A9.

```
• DevSecOps-Tp1 $ export NVD_API_KEY="2F1F672D-F4D0-453D-825E-7E9BE747F8A9"
```

4.2 Dépendances du projet

Le fichier `requirements.txt` déclare notamment `**PyYAML==5.3.1**`, version connue pour exposer une CVE critique.

Le scan SCA ci-dessous analyse ce fichier avec OWASP Dependency-Check.

4.3 Scan SCA - AVANT mise à jour

```
mkdir -p reports .odc-data

docker run --rm \
  -v "$PWD":/src \
  -v "$PWD/reports":/report \
  -v "$PWD/.odc-data":/usr/share/dependency-check/data \
  owasp/dependency-check \
  --scan /src \
  --enableExperimental \
  --format HTML --format JSON \
  --out /report \
  --project vulnerable-notes \
  --nvdApiKey "$NVD_API_KEY"
```

```
○ DevSecOps-Tp1 $
docker run --rm \
  -v "$PWD":/src \
  -v "$PWD/reports":/report \
  -v "$PWD/.odc-data":/usr/share/dependency-check/data \
  owasp/dependency-check \
  --scan /src \
  --enableExperimental \
  --format HTML --format JSON \
  --out /report \
  --project vulnerable-notes \
  --nvdApiKey "$NVD_API_KEY"
Unable to find image 'owasp/dependency-check:latest' locally
latest: Pulling from owasp/dependency-check
6de38e210efd: Download complete
0b81a78f4d17: Downloading [=====>] 24.12MB/45.11MB
e6f31ffc071e: Pull complete
4f4fb700ef54: Already exists
24cad73030d: Downloading [=====>] 26.21MB/32.27MB
bcd962b17ba: Pull complete
751d9cdba640: Pull complete
63412a635afe: Downloading [=>] 2.097MB/74.56MB
14bdcc1aee9f: Pulling fs layer
```

```
[INFO] Completed processing batch 198/202 (98%) in 78ms
[INFO] Completed processing batch 199/202 (99%) in 79ms
[INFO] Completed processing batch 200/202 (99%) in 87ms
[INFO] Completed processing batch 201/202 (100%) in 56ms
[INFO] Completed processing batch 202/202 (100%) in 28ms
[INFO] Updating CISA Known Exploited Vulnerability list: https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json
[INFO] Begin database defrag
[INFO] End database defrag (2523 ms)
[INFO] Check for updates complete (374909 ms)
[INFO]
```

Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition, and there are NO warranties, implied or otherwise, with regard to the analysis or its use. Any use of the tool and the reporting provided is at the user's risk. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the resulting report.

About ODC: <https://dependency-check.github.io/DependencyCheck/general/internals.html>
False Positives: <https://dependency-check.github.io/DependencyCheck/general/suppression.html>

```
[INFO] Analysis Started
[INFO] Finished File Name Analyzer (0 seconds)
[INFO] Finished Python Package Analyzer (0 seconds)
[INFO] Finished pip Analyzer (0 seconds)
[INFO] Finished Dependency Merging Analyzer (0 seconds)
[INFO] Finished Hint Analyzer (0 seconds)
[INFO] Created CPE Index (0 seconds)
[INFO] Finished NPM CPE Analyzer (0 seconds)
[INFO] Created CPE Index (0 seconds)
[INFO] Finished CPE Analyzer (0 seconds)
[INFO] Finished False Positive Analyzer (0 seconds)
[INFO] Finished NVD CVE Analyzer (0 seconds)
[WARN] Sonatype OSS Index Analyzer disabled due to missing credentials. Authentication with token is now required, and OSS Index is migrating to Sonatype Guide. See https://dependency-check.github.io/DependencyCheck/analyzers/oss-index-analyzer.html for more information on authentication with Sonatype Guide OSS Index.
[INFO] Finished Vulnerability Suppression Analyzer (0 seconds)
[INFO] Finished Known Exploited Vulnerability Analyzer (0 seconds)
[INFO] Finished Dependency Bundling Analyzer (0 seconds)
[INFO] Finished Unused Suppression Rule Analyzer (0 seconds)
[INFO] Analysis Complete (1 seconds)
[INFO] Writing HTML report to: /report/dependency-check-report.html
[INFO] Writing JSON report to: /report/dependency-check-report.json
```

Rapports sauvegardés :

[reports/dependency-check-avant.html] (reports/dependency-check-avant.html) ·
[reports/dependency-check-avant.json] (reports/dependency-check-avant.json)



Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition, and there are NO warranties, implied or otherwise, with regard to the analysis or its use. Any use of the tool and the reporting provided is at the user's risk. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the resulting report.

[How to read the report](#) | [Suppressing false positives](#) | [Getting Help: github issues](#)

Project: vulnerable-notes

Scan Information ([show all](#)):

- *dependency-check version:* 13.0.0
- *Report Generated On:* Wed, 7 Oct 2026 23:11:12 GMT
- *Dependencies Scanned:* 7 (7 unique)
- *Vulnerable Dependencies:* 1
- *Vulnerabilities Found:* 1
- *Vulnerabilities Suppressed:* 0
- ...

Summary

Summary of Vulnerable Dependencies ([click to show all](#))

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
PyYAML:5.3.1	cpe:2.3:a:pyyaml:pyyaml:5.3.1:*:*:*:*:* cpe:2.3:a:yaml_project:yaml:5.3.1:*:*:*:*:*	pkg:pypi/pyyaml@5.3.1	CRITICAL	1	Low	3

Dependencies (vulnerable)

PyYAML:5.3.1

File Path: /src/requirements.txt:PyYAML/5.3.1

Evidence

Identifiers

- pkg:pypi/pyyaml@5.3.1 (Confidence: Highest)
- cpe:2.3:a:pyyaml:pyyaml:5.3.1:*:*:*:*:* (Confidence: Low) suppress
- cpe:2.3:a:yaml_project:yaml:5.3.1:*:*:*:*:* (Confidence: Low) suppress

Published Vulnerabilities

CVE-2020-14343 suppress

A vulnerability was discovered in the PyYAML library in versions before 5.4, where it is susceptible to arbitrary code execution when it processes untrusted YAML files through the `full_load` method or with the `FullLoader` loader. Applications that use the library to process untrusted input may be vulnerable to this flaw. This flaw allows an attacker to execute arbitrary code on the system by abusing the `python/object/new` constructor. This flaw is due to an incomplete fix for CVE-2020-1747.

CWE-20 Improper Input Validation

CVSSv3:

- Base Score: CRITICAL (9.8)
- Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:A/H/E:P/RC:R/MAV:A

CVSSv2:

- Base Score: HIGH (10.0)
- Vector: /AV:N/AC:L/Au:N/C:C/I:C/A:C

References:

- af854a3a-2127-422b-91ae-364da2661108 - <https://github.com/SeldonIO/seldon-core/issues/2252>
- af854a3a-2127-422b-91ae-364da2661108 - <https://github.com/yaml/pyyaml/issues/420>
- af854a3a-2127-422b-91ae-364da2661108 - [ISSUE TRACKING,THIRD PARTY ADVISORY](#)
- af854a3a-2127-422b-91ae-364da2661108 - [PATCH,THIRD PARTY ADVISORY](#)
- af854a3a-2127-422b-91ae-364da2661108 - [PATCH,THIRD PARTY ADVISORY](#)
- secalert@redhat.com - <https://github.com/SeldonIO/seldon-core/issues/2252>
- secalert@redhat.com - <https://github.com/yaml/pyyaml/issues/420>
- secalert@redhat.com - [ISSUE TRACKING,THIRD PARTY ADVISORY](#)
- secalert@redhat.com - [PATCH,THIRD PARTY ADVISORY](#)
- secalert@redhat.com - [PATCH,THIRD PARTY ADVISORY](#)

Vulnerable Software & Versions: [\(show all\)](#)

- [cpe:2.3:a:pyyaml:pyyaml:*:*:*:*:* versions from \(including\) 5.1: versions up to \(excluding\) 5.4](#)
- ...

4.4 Vulnérabilité identifiée

Champ	Valeur
Dépendance	PyYAML 5.3.1
CVE	CVE-2020-14343
Sévérité	CRITICAL
Problème	Exécution de code arbitraire lors du traitement de YAML non fiable (<code>full_load</code> / <code>Loader</code> non sûr) dans les versions < 5.4
Remédiation	Mettre à jour vers une version corrigée

Pourquoi mettre à jour est nécessaire : on ne « patch » pas soi-même une bibliothèque tierce de façon fiable. La correction maintenue par le projet amont (nouvelle version) est la remédiation standard en SCA.

4.5 Correctif SCA

```
1 Flask==2.0.3
2 Werkzeug==2.0.3
3 Jinja2==3.0.3
4 MarkupSafe==2.0.1
5 itsdangerous==2.0.1
6 click==8.0.4
7 PyYAML==5.3.1
8 PyYAML==6.0.2
```

```
# avant
PyYAML==5.3.1

# après
PyYAML==6.0.2
```

4.6 Scan SCA - APRÈS mise à jour

Même commande Dependency-Check, puis renommage des rapports en dependency-check-apres.*.



Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition, and there are NO warranties, implied or otherwise, with regard to the analysis or its use. Any use of the tool and the reporting provided is at the user's risk. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the resulting report.

[How to read the report](#) | [Suppressing false positives](#) | Getting Help: [github issues](#)

Project: vulnerable-notes-fixed

Scan Information ([show all](#)):

- *dependency-check version:* 13.0.0
- *Report Generated On:* Wed, 7 Oct 2026 23:34:41 GMT
- *Dependencies Scanned:* 7 (7 unique)
- *Vulnerable Dependencies:* 0
- *Vulnerabilities Found:* 0
- *Vulnerabilities Suppressed:* 0
- ...

Summary

Summary of Vulnerable Dependencies ([click to show all](#))

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
------------	-------------------	---------	------------------	-----------	------------	----------------

Dependencies (vulnerable)

This report contains data retrieved from the [National Vulnerability Database](#).
This report may contain data retrieved from the [CISA Known Exploited Vulnerability Catalog](#).
This report may contain data retrieved from the [Github Advisory Database \(via NPM Audit API\)](#).
This report may contain data retrieved from [RetireJS](#).
This report may contain data retrieved from the [Sonatype Guide OSS Index API](#).

Rapports :

reports/dependency-check-apres.html ·
reports/dependency-check-apres.json

Résultat : pkg:pypi/pyyaml@6.0.2 → **0 vulnérabilité** (CVE-2020-14343 absente).

5. Démonstration et synthèse

5.1 Avant / après

Contrôle	Avant	Après
Bandit	3 issues Medium (B608, B704, B104)	No issues identified
curl SQLi	Tous les users renvoyés	Charge traitée comme un nom littéral (pas d'injection)

Contrôle	Avant	Après
curl XSS	<script> brut	Contenu échappé (<script>)
Dependency-Check	PyYAML 5.3.1 → CVE-2020-14343 CRITICAL	PyYAML 6.0.2 → 0 CVE

5.2 Ce que le TP a permis d'apprendre

1. **SAST** trouve des motifs dangereux **dans le code** (sans forcément exécuter l'app).
2. **SCA** trouve des failles **publiées** dans les dépendances (CVE / NVD).
3. Les deux sont complémentaires : du code « propre » avec une lib vulnérable reste dangereux, et l'inverse aussi.
4. La preuve attendue en DevSecOps : **scan** → **correctif** → **re-scan**.

Conclusion

Le TP valide les objectifs du sujet :

- approche **shift-left** appliquée avec Docker + scanners locaux ;
- **au moins deux** vulnérabilités SAST (injection SQL, XSS) identifiées, corrigées et absentes du second rapport Bandit ;
- **au moins une** dépendance vulnérable (PyYAML / CVE-2020-14343) mise à jour, avec disparition de l'alerte SCA.